

Introducción al desarrollo de aplicaciones

Este documento introduce los conceptos básicos relativos al desarrollo de aplicaciones informáticas. Comenzaremos describiendo el sistema informático haciendo hincapié en la relación con la ejecución de aplicaciones. Posteriormente veremos los lenguajes de programación, diferentes clasificaciones de los mismos y los diferentes estados de un programa en función de su código. Finalmente veremos las fases del desarrollo software.

Introducción al desarrollo de aplicaciones

Copyright © 2023 by Rafael Lozano Luján.

Este documento está sujeto a derechos de autor. Todos los derechos están reservados por el Autor de la obra, ya sea en su totalidad o en parte de ella, específicamente los derechos de:

- La reproducción total o parcial mediante fotocopia, escaneo o descarga.
- La distribución o publicación de copias a terceros.
- La transformación mediante la modificación, traducción o adaptación que altere la forma de la obra hasta obtener una diferente a la original.

Tabla de contenido

1	Introducción.....	1
2	El sistema informático.....	1
2.1	Interacción del software con el hardware.....	3
2.1.1	Arquitectura Von Neumann.....	3
2.1.2	Ejecución de una instrucción.....	5
3	Lenguajes de programación.....	7
3.1	Clasificación de los lenguajes de programación.....	8
3.1.1	Según su evolución histórica.....	9
3.1.1.1	Lenguajes de bajo nivel.....	9
3.1.1.2	Lenguajes de nivel medio.....	11
3.1.1.3	Lenguajes de alto nivel.....	11
3.1.2	Según la forma de ejecución.....	12
3.1.3	Según el paradigma de programación.....	13
3.2	Traducción de código fuente.....	14
3.2.1	Intérprete.....	15
3.2.2	Compilador.....	15
3.2.3	Código intermedio y máquina virtual.....	16
3.2.4	Biblioteca.....	17
3.3	Código fuente, objeto y ejecutable.....	20
4	Desarrollo de aplicaciones.....	21
4.1	Ciclo de vida del software.....	21
4.1.1	Planificación.....	21
4.1.2	Análisis.....	22
4.1.3	Diseño.....	22
4.1.4	Codificación.....	23
4.1.5	Pruebas.....	23
4.1.6	Despliegue o instalación.....	23
4.1.7	Documentación.....	23
4.1.8	Mantenimiento.....	24
4.2	Herramientas de desarrollo.....	24
5	Bibliografía.....	27

Introducción al desarrollo de aplicaciones

1 Introducción

En este documento vamos a aprender a reconocer elementos y herramientas que se usan para desarrollar un programa informático, así como las características y fases que tiene que pasar hasta su puesta en funcionamiento.

Empezaremos a ver qué la relación existente dentro de un sistema informático entre el hardware y el software, de forma tal que se detallará como los diferentes componentes del hardware interactúan con el software y, más concretamente, con la ejecución de las instrucciones de una aplicación.

Posteriormente veremos en qué consiste el proceso de desarrollo de una aplicación informática abordando las fases por las que hay que pasar, más conocido como el ciclo de vida del software. También veremos los distintos tipos de lenguajes de programación que podremos utilizar, sus características y las distintas clasificaciones, junto con los diferentes estados en que se encuentran las instrucciones de un programa.

2 El sistema informático

Una definición formal de sistema informático podría ser la siguiente:

Conjunto de elementos que interactúan entre sí para realizar el tratamiento automático de la información.

El sistema informático no está ligado a ningún tipo específico de dispositivo electrónico. Puede ser desde un pequeño teléfono móvil hasta una red de comunicaciones de área extensa (Internet). Un sistema informático está constituido básicamente por los siguientes componentes:

- ✓ Parte física o hardware.
- ✓ Parte lógica o software.
- ✓ Las personas.

Se denomina hardware al conjunto de dispositivos físicos que integran el ordenador, como por ejemplo la CPU, el teclado, la impresora, el monitor, etc. Está formado por el ordenador y todo lo que se conecta a él.

Software es el conjunto de órdenes que ejecutan los distintos componentes del sistema informático para que realicen las distintas tareas que tienen asignadas. El elemento principal del software es el programa o aplicación.

Un programa o aplicación es un conjunto finito de instrucciones que realizan una tarea concreta.

El concepto de software va más allá de los programas en sus distintos estados: código fuente, binario o ejecutable; también su documentación, los datos a procesar e incluso la información de usuario forman parte del software: es decir, abarca todo lo intangible, todo lo «no físico» relacionado con el sistema informático.

El software y el hardware evolucionan rápidamente; la evolución del hardware está orientada a conseguir máquinas, cada vez más rápidas, que sean capaces de procesar y almacenar más cantidad de información. La evolución del software está orientada al desarrollo de nuevas aplicaciones o programas que aprovechen mejor el hardware existente.

Las personas son las encargadas de redactar las órdenes con las que el ordenador va a ejecutar las acciones que se desee o bien aquellas otras que, disponiendo de un ordenador, realizan con él determinados trabajos de gestión, técnicos o científicos. Con este personal humano pueden hacerse dos grupos: el técnico informático y el usuario. El primero abarca un amplio abanico de profesionales informáticos que se encargan de las más diversas tareas: analistas de sistemas, administradores de sistemas, programadores, diseñadores de interfaces de usuario, reparación de dispositivos digitales, diseñadores de bases de datos, ... Básicamente, el técnico informático se ocupa de que un sistema esté en perfecto estado operativo para los usuarios, los cuales utilizan los ordenadores en sus rutinas diarias mediante el empleo de alguna aplicación software: procesadores de texto, aplicaciones de diseño gráfico, realización de cálculos científicos, etc.

Finalmente, la información constituye el conjunto de datos que son de interés al usuario. Para que estos datos puedan introducirse, almacenar y procesarse en el ordenador tienen que estar representados en una forma entendible por el mismo.

2.1 Interacción del software con el hardware

En el apartado anterior vimos lo que es hardware y software. Ambos son entidades separadas, pero estrechamente vinculadas, ya que uno sin el otro no pueden existir. El elemento principal del software, es decir, las aplicaciones o programas interactúan con el hardware para realizar lo que se conoce como el proceso de la información.

Los ordenadores son máquinas y, como tales, están constituidos por elementos físicos; sin embargo, estos componentes no constituyen por sí mismos un ordenador, sino que necesitan una serie de instrucciones que les hagan funcionar. Como hemos dicho anteriormente, el ordenador procesa información y lo hace tomando datos, o información aún no procesada, como punto de partida. Procesados esos datos por medio de un conjunto de órdenes o instrucciones, (programa), se convierten en información elaborada o resultados. Este proceso recibe en informática el nombre de tratamiento de la información o proceso de la información.

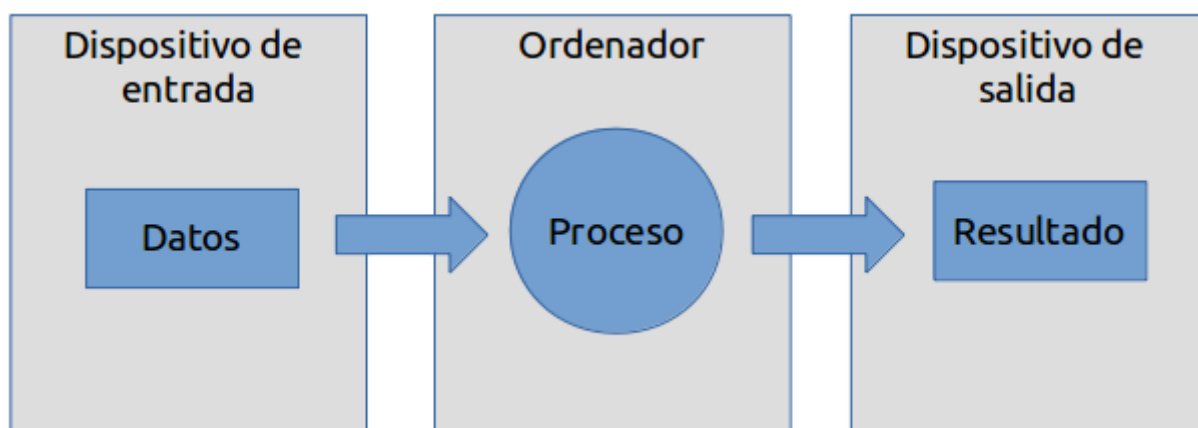


Figura 1.- Proceso de la información

Según todo lo dicho anteriormente, las principales operaciones que un ordenador puede realizar sobre la información son las siguientes:

- ✓ Entrada de datos.- Consiste en suministrar datos al ordenador para realizar algún tipo de tratamiento sobre los mismos.
- ✓ Salida de datos.- Obtener información de un ordenador. Normalmente la información obtenida es el resultado de algún proceso realizado sobre los datos de entrada.
- ✓ Proceso.- Cualquier operación sobre la información que implique una transformación de la misma. Estas pueden ser: ordenación, selección, combinación, etc.

A cada operación es llevada a cabo por un componente físico del ordenador, por ejemplo, los datos se introducen a través del teclado. En el proceso intervienen la memoria, procesador, el software, etc. La salida de la información se realiza a la pantalla o por la impresora.

2.1.1 Arquitectura Von Neumann

En los inicios de la informática, los primeros ordenadores que se desarrollaron fueron

de programa fijo, lo que significa que solamente ejecutaban un programa. Cambiar el programa implicaba un rediseño del ordenador con una reestructuración y recableado de la máquina que llevaba mucho tiempo y que en la práctica suponía la construcción de un nuevo ordenador.

Surgió entonces la necesidad de que los ordenadores pudieran ejecutar cualquier programa sin necesidad de tener que rediseñar la máquina. Para este propósito, el matemático y físico John Von Neumann diseñó y describió en 1945 la arquitectura de un ordenador, denominada **ordenador de programa almacenado**, según la cual la tarea que realiza un ordenador es controlada por un programa almacenado en una memoria de acceso aleatorio. Además, el ordenador no es un ente único, sino que está formado por diferentes partes cada una con una tarea muy concreta. Son estas:

1. Unidad central de proceso → El encargado de ejecutar las instrucciones de un programa. Este a su vez dispone de los siguientes componentes:
 - a) Unidad Aritmético-Lógica → Encargada de realizar las operaciones.
 - b) Unidad de control → Encargada de decodificar qué operación indica la instrucción del programa.
 - c) Registros de la CPU → Almacenan datos necesarios para ejecutar las instrucciones. Algunos de ellos tienen una finalidad muy concreta y estrechamente relacionada con la ejecución de instrucciones. Los más relevantes son:
 - Registro Dirección de Memoria (MAR) → Se emplea para acceder a memoria. Almacena la dirección de memoria desde la que se lee, o escribe, una instrucción del programa o dato.
 - Registro Intercambio de Memoria (MIR) → Se emplea para depositar un dato leído desde memoria o un dato a guardar en memoria.
 - Contador del programa (PC) → Contiene la dirección de memoria de la siguiente instrucción a ejecutar.
 - Registro Instrucción (IR) → Se emplea para depositar la siguiente instrucción a ejecutar de un programa que se ha leído desde memoria.
 - Registros de datos → Se almacenan datos que emplea la ejecución de instrucciones.
 - d) Bus interno → Líneas de comunicación entre los componentes internos de la CPU.
2. Memoria principal.- Se encarga de almacenar tanto las instrucciones de un programa en ejecución como los datos que manejan esas instrucciones.
3. Almacenamiento externo.- Para almacenar de forma permanente los resultados del proceso de la información.
4. Dispositivos de entrada y salida.- Para comunicar al ordenador con el exterior.

5. Buses o líneas de comunicación entre los componentes anteriores. Principalmente son los siguientes:
 - a) Bus de direcciones → Se transmite la dirección de memoria a la que se quiere acceder.
 - b) Bus de datos → Se transmiten datos de memoria a CPU o viceversa.
 - c) Bus de control → Se activan señales para que un componente realiza una tarea concreta.

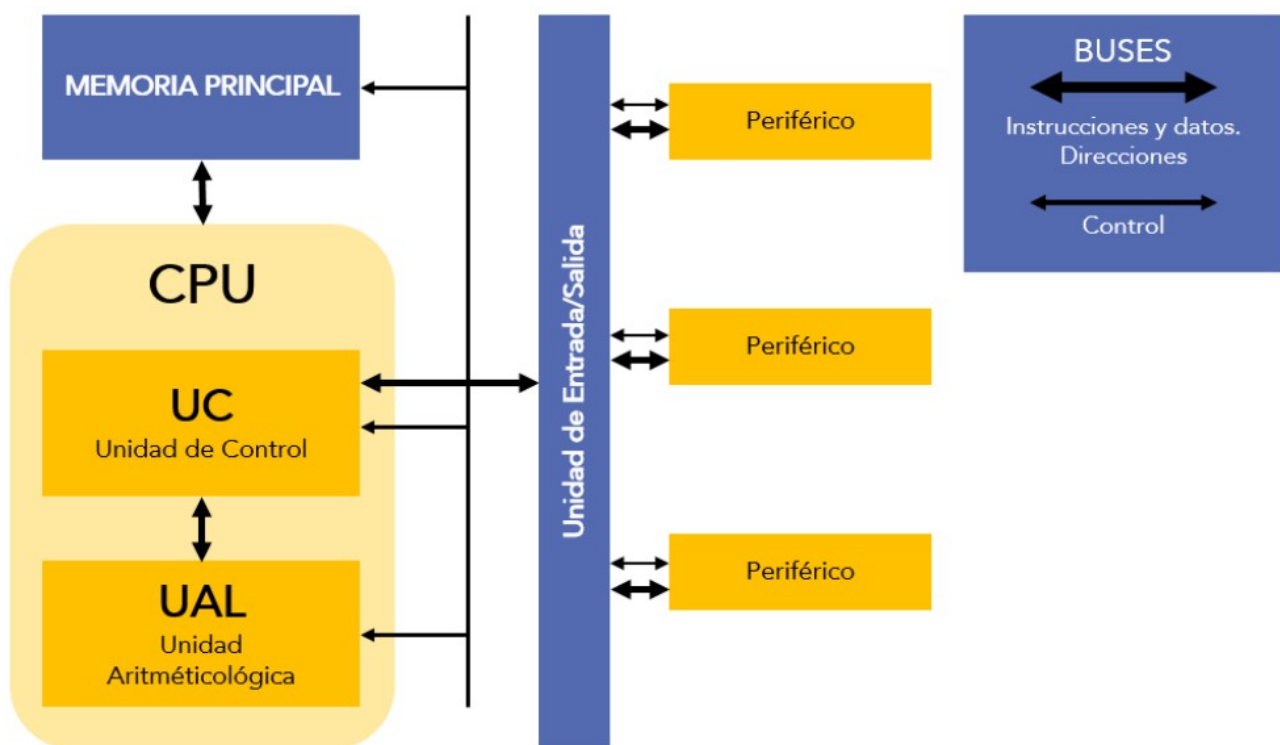


Figura 2.- Arquitectura Von Neumann

La anterior figura muestra un diagrama de la arquitectura de Von Neumann, que resulta independiente del tipo de ordenador.

2.1.2 Ejecución de una instrucción

Cuando el usuario invoca la ejecución de un programa, éste se encuentra habitualmente almacenado en el almacenamiento externo (disco duro o cualquier otro dispositivo para tal fin). Desde aquí se carga en la memoria principal del ordenador. La carga supone crear una copia, que puede ser no completa, del programa en la memoria principal. Una vez ahí la CPU comienza a ejecutar las instrucciones del programa una a una y en orden secuencial. Básicamente ocurren los siguientes eventos:

1. Carga de la siguiente instrucción a ejecutar desde la memoria principal a uno de los registros de la CPU. Esta carga supone: La dirección de memoria de la siguiente instrucción a ejecutar se encuentra en el registro PC. Este valor se copia en el registro MAR el cual se coloca en el bus de direcciones para acceder a memoria. Desde aquí

- se copia la instrucción en el registro MIR que transmite la instrucción leída por el bus de datos desde la memoria principal al registro IR.
2. Se decodifica la instrucción a ejecutar y los datos implicados en la misma. Esta fase la realiza la Unidad de Control y consiste en identificar qué operación indica la instrucción.
 3. Se cargan los datos que la instrucción necesita para su ejecución desde la memoria principal a los registros de datos en la CPU. Por cada dato hay que depositar su dirección de memoria en el registro MAR que se coloca en el bus de direcciones, acceder a memoria y copiar el dato en el registro MIR para su transmisión por el bus de datos hasta un registro de datos en la CPU. Si estos datos no estuvieran en la memoria principal, se tienen que cargar desde el almacenamiento externo a la memoria principal y, después, se cargan en los registros de la CPU. La carga se realiza transmitiendo los datos desde la memoria principal a los registros de la CPU por el bus, uno a uno.
 4. Se ejecuta la operación indicada en la instrucción por la ALU con los datos necesarios. La Unidad de Control activa las señales necesarias para que la ALU ejecute la operación. El resultado de la operación se almacena en un registro de datos la CPU.
 5. Se copia el resultado de la operación en la memoria principal. El resultado se copia desde el registro de datos de la CPU al registro MIR conectado al bus de direcciones. Se copia en el registro MAR la dirección de memoria donde se tiene que almacenar el resultado. Se transmite el contenido del registro MAR por el bus de datos a la memoria principal.

El proceso anterior se repite para la siguiente y sucesivas instrucciones del programa. Al ejecutar la última instrucción del programa, éste se borra de la memoria principal para dejar espacio para la ejecución de otros programas.

A modo de ejemplo vamos a realizar la ejecución de una instrucción simple de un modo conceptual. Imaginemos que tenemos un programa extremadamente sencillo que pide por teclado dos números y los suma. La instrucción que realizará la operación de nuestro programa se podría corresponder con la siguiente línea:

```
a = b + c
```

Supondremos que el programa (instrucciones y datos) ya están cargados en la memoria principal cuando el usuario ha invocado su ejecución. Por tanto, los valores de los datos **b** y **c** están cargados en la memoria principal. Además, debemos tener en cuenta una serie de reglas intrínsecas del sistema:

- ✓ La ALU solo puede ejecutar una operación a la vez.
- ✓ Los registros de la CPU solo pueden almacenar un dato a la vez.
- ✓ Por el bus solo pueden transmitirse un dato a la vez.

Según lo anterior la sucesión de operaciones para ejecutar esta instrucción serían:

1. Se transmite por el bus la instrucción a un registro de la CPU.
2. La UC identifica e interpreta la instrucción. Resuelve que es una suma de dos datos.
3. Se transmite por el bus el dato **b** desde la memoria principal a un registro en la CPU.
4. Se transmite por el bus el dato **c** desde la memoria principal a un registro en la CPU.
5. La ALU realiza la suma de **b** y **c**. El resultado se almacena en un registro de la CPU.
6. Se transmite por el bus el resultado de la operación a la memoria principal generándose el dato **a**.

Este proceso está muy simplificado para facilitar su comprensión. En realidad es más complejo. La unidad de control se encarga de marcar la pauta en la ejecución de las microinstrucciones y de activar las señales para que cada componente realice su función, todo ello regido por el reloj de la CPU. Cada dato en memoria tiene una dirección para su rápida localización, etc.

3 Lenguajes de programación

Los ordenadores son máquinas complejas y sofisticadas. No obstante desconocen la cuestión más sencilla que se les pueda plantear. El hardware por sí solo no es suficiente para hacer operativo a los ordenadores, debido a que precisan de órdenes que los hagan funcionar y estas órdenes forman lo que se denomina un programa. La preparación de las diferentes operaciones ordenadas que forman nuestro programa necesita una representación detallada para reflejar la secuencia de ejecución de las mismas, para lo cual se utilizan los algoritmos.

La determinación exacta y sin ambigüedades de la secuencia de pasos elementales que deben ejecutarse para encontrar la solución de un problema.

Un algoritmo debe ser:

- ✓ Preciso → En cuanto al orden de las operaciones.
- ✓ Finito → En cuanto al número de operaciones.
- ✓ Correcto → Debe conducir a la solución del problema.

No existe una única solución para un problema, de ahí que un mismo problema puede resolverse por más de un algoritmo.

Un algoritmo puede expresarse mediante diferentes formas:

- ✓ Pseudocódigo → Es una codificación simbólica, mediante lenguajes nemotécnicos, parecidos al lenguaje natural, que no pueden ser ejecutados por el ordenador pero que son entendibles por cualquier persona. Mediante este lenguaje formal se pueden

especificar las operaciones que forman el algoritmo.

- ✓ **Lenguajes de programación** → Es la expresión del algoritmo para que pueda ser almacenado, leído y ejecutado por un ordenador. Sólo es entendible por aquél que conozca el lenguaje de programación.
- ✓ **Diagramas de flujo u Ordinograma** → Es la representación gráfica de un algoritmo. Los pasos a realizar para expresar la solución de un problema son componentes gráficos enlazados entre sí para denotar orden en la ejecución de las operaciones. Representan gráficamente la secuencia lógica de las operaciones que realiza el programa. Su desventaja es que esta representación puede ser muy grande incluso para algoritmos muy pequeños.

Como acabamos de ver, una de las formas de expresar las instrucciones de un programa es mediante un lenguaje de programación.

Un lenguaje de programación es un conjunto de símbolos, reglas sintácticas y semánticas que definen su estructura y el significado de sus elementos y expresiones, con los cuales el programador puede especificar un proceso que se realiza a los datos.

Mediante el lenguaje de programación se expresa el procesamiento a llevar a cabo por el ordenador. Por tanto, se emplea para crear programas que controlan el comportamiento físico y lógico de una máquina, expresando la solución a un problema con precisión. Un lenguaje de programación proporciona los elementos necesarios para traducir los pasos de un algoritmo a instrucciones comprensibles para la máquina.

3.1 Clasificación de los lenguajes de programación

Existen varios criterios a la hora de realizar una clasificación de los lenguajes de programación. Generalmente son:

- ✓ **Evolución histórica** → Los lenguajes evolucionan con el tiempo para facilitar al programador la tarea de programar.
- ✓ **Nivel de abstracción** → Un lenguaje puede estar enfocado al hardware del ordenador o acercarse más al lenguaje natural abstrayendo la complejidad de la máquina.
- ✓ **Paradigma de programación** → Existen varios modelos de programación que disponen de lenguajes adaptados a cada modelo en particular.
- ✓ **Propósito del programa** → Las aplicaciones informáticas se aplican en todas las disciplinas del saber. Cálculo científico, bases de datos, videojuegos, multimedia, etc. Algunos lenguajes son más adecuados que otros según el tipo de programa que se vaya a realizar.

Los dos primeros criterios están estrechamente relacionados. La evolución histórica de los lenguajes de programación han permitido evolucionar los lenguajes desde la codificación en binario (su nivel más bajo) al lenguaje natural (su nivel más alto) para simplificar la tarea de programar. Los lenguajes de programación se pueden clasificar en las

siguientes categorías.

3.1.1 Según su evolución histórica

Se suele decir que los lenguajes de programación tienen niveles o “generaciones” según sus primitivas sean de más o menos nivel, en el sentido de que tengan más o menos cualidades para acercarse al lenguaje humano y de que sean capaces o no de realizar acciones más complejas con sus primitivas. Cada lenguaje, según su nivel, tiene sus ventajas e inconvenientes y han sido desarrollados para fines concretos. Para ver una evolución histórica de los lenguajes de programación podemos visitar <https://www.timetoast.com/timelines/generaciones-de-los-lenguajes-de-programacion-78279214-40c7-4337-be29-0bcd0e376bd2>.

3.1.1.1 Lenguajes de bajo nivel

El lenguaje de más bajo nivel es el lenguaje máquina. Un ordenador solo entiende un lenguaje binario que consta de 0 y 1, denominados bits o dígitos binarios. Por tanto un programa expresado en este lenguaje solamente está compuesto por instrucciones escritas con 0s y 1s. Este lenguaje es muy complicado de usar para una persona, ya que trabajar con unos y ceros resulta lento, tedioso y es propenso a errores. En los inicios de la informática, esta era la forma inicial de escribir los programas. Cada programa se diseñaba y construía para un ordenador concreto. Las primitivas del lenguaje ejecutaban acciones simples y concretas. En los lenguajes de bajo nivel se dice que están orientados a la máquina.

Este lenguaje es específico del procesador donde se ejecutará el programa. Por tanto requiere de un conocimiento exhaustivo del hardware de la máquina. La siguiente imagen muestra un ejemplo de programa escrito en lenguaje máquina que realiza tres sumas de números almacenados en memoria.

Memoria

1	10101011 00001010 00001011 00001101
2	10101011 00001101 00001100 00001101
3	00001110 00001101 00000011 00001101
4	00000000 00000000 00000000 00000000
5	
6	
7	
8	

Figura 3: Programa en lenguaje máquina

De esta forma el programa más sencillo se convertía en algo complejo y tedioso. La dificultad de detectar y corregir un error se añadía a la dificultad de programar. La situación no tardó en cambiar con la aparición de los llamados lenguajes ensambladores.

Un intento de facilitar la tarea de programación fue el lenguaje ensamblador. En este, las instrucciones de un programa se codifican simbólicamente mediante códigos mnemotécnicos en vez de hacerlo directamente con su representación binaria. Estos

códigos mnemotécnicos son más fáciles de recordar que sus equivalentes en binario y no solo expresan las instrucciones del programa, sino también los datos que utiliza, las direcciones de memoria donde se almacenan, los códigos de operación, etc.

Entre las características de este lenguaje están:

- ✓ En el lenguaje ensamblador una instrucción primitiva se corresponde con una instrucción máquina, por tanto cada instrucción en ensamblador se traduce a una instrucción máquina.
- ✓ Las direcciones de memoria no se referencian explícitamente, sino simbólicamente. En ensamblador, al añadir o eliminar una instrucción no implica que el programador tenga que cambiar la dirección de las siguientes instrucciones ya que el procesador hará el cálculo de direcciones de memoria, lo que facilita cierta flexibilidad en la programación.
- ✓ La ejecución de programas es muy rápida, más que en cualquier lenguaje de mayor nivel.
- ✓ Permite la explotación total de los recursos de la máquina.
- ✓ Un lenguaje ensamblador es específico de un procesador determinado, el cual contempla un conjunto concreto de instrucciones. Por eso se habla de lenguajes ensambladores, en plural: para Intel 386, Motorola 68000, DLX, ARC, etc.). Debido a ello, el programador requiere buenos conocimientos de la forma de funcionamiento del procesador específico (conjunto de instrucciones).
- ✓ La escritura de programas es larga, difícil y propensa a errores, aunque siempre será mejor que programar en lenguaje máquina.

Solamente cuando escribimos un programa con lenguaje máquina es directamente ejecutable en el ordenador ya que se emplea el propio lenguaje que entiende el procesador. Sin embargo, un programa escrito en lenguaje ensamblador no es ejecutable por el ordenador. Se necesita traducir a código máquina y esta traducción la realiza un programa denominado también ensamblador.

La siguiente imagen muestra un fragmento de un programa escrito en lenguaje ensamblador.

```

.model small
.stack 100h
.data
    _var1 db 0
    _var2 db 0
    _msg_true db 10,13, "Condicion verdad$"
    _msg_false db 10,13, "Condicion falsa$"
.code
.startup

mov ah, 01h
int 21h
sub al, 30h
mov _var1, al

mov ah, 01h
int 21h
sub al, 30h
mov _var2, al

; if code (var1 < var2)
mov bl, _var2
cmp _var1, bl
jl _if_true
; false
lea dx, _msg_false
mov ah, 09h
int 21h
jmp _if_end

_if_true:
lea dx, _msg_true
mov ah, 09h
int 21h
jmp _if_end

```

Figura 4: Ejemplo programa en lenguaje ensamblador para el procesador Intel 8086

3.1.1.2 Lenguajes de nivel medio

El lenguaje de nivel medio es aquel que combina características tanto de los lenguajes de bajo nivel como de los lenguajes de alto nivel. Básicamente, se trata de un lenguaje que permite al programador realizar tareas de programación de manera más eficiente y con mayor control que en los lenguajes de alto nivel, pero sin la complejidad que implica el uso de lenguajes de bajo nivel. Estos lenguajes se consideran lenguajes de 3ª generación.

En el caso del lenguaje C, este es considerado un lenguaje de nivel medio debido a que combina elementos propios de los lenguajes de bajo nivel –como el acceso directo a los recursos del sistema operativo y la memoria– con elementos de los lenguajes de alto nivel –como la sintaxis simplificada y la portabilidad–. Esto permite a los programadores trabajar con mayor precisión y eficiencia en tareas que requieren un mayor control sobre los recursos del sistema, pero sin tener que preocuparse por detalles de bajo nivel que pueden resultar tediosos y propensos a errores.

Un programa escrito con este tipo de lenguajes también necesita una traducción a código binario el cual realiza mediante un programa traductor que puede ser un intérprete o un compilador, los cuales se verán en un apartado posterior.

3.1.1.3 Lenguajes de alto nivel

Los lenguajes de alto nivel permiten programar sin conocer el funcionamiento interno de la máquina ya que las sentencias se acercan mucho al lenguaje natural. Sin embargo, los programas escritos con alguno de estos lenguajes no son directamente ejecutables por el procesador. Es necesario traducirlo a lenguaje máquina para poder ejecutarse. La traducción

se realiza mediante unas aplicaciones denominadas traductores, de los cuales hay dos tipos principales: compiladores e intérpretes. Más adelante se tratará en detalle la traducción de un programa escrito en lenguaje de alto nivel.

En los lenguajes de alto nivel no hablamos de instrucciones, sino de sentencias. Una sentencia en un lenguaje de alto nivel equivale a muchas instrucciones en lenguaje máquina cuando se traduce. Así, una sentencia simple en lenguaje de alto nivel que calcula una expresión matemática equivale a decenas de instrucciones máquina. Cualquier aplicación de tamaño mediano escrita en cualquier lenguaje de alto nivel acaba siendo, miles o centenares de miles de instrucciones en lenguaje máquina.

Los lenguajes de programación de alto nivel han venido surgiendo desde los años 60. Cada lenguaje nace para dar respuesta a la necesidad de programar soluciones en campos concretos (aplicaciones científicas, de gestión, etc.) y la evolución de cada uno de ellos ha ocasionado un gran número de versiones.

Entre estos están los lenguajes de 4ª y 5ª generación. Cada generación pretende proporcionar un mayor nivel de abstracción de los detalles internos del hardware del ordenador, haciendo más potente, versátil y amigable la tarea de programar.

Estos lenguajes incluyen características avanzadas de programación como reutilización de código, operar sobre grandes cantidades de información, soporte a gestión de bases de datos, generación de informes, optimización matemática, desarrollo de GUIs y desarrollo web.

Además, existen ciertas herramientas que permiten construir aplicaciones sencillas combinando piezas prefabricadas: acceso a base de datos, capacidades gráficas, generación de código automáticamente, así como poder programar visualmente. Estas herramientas no son propiamente hablando un lenguaje de programación. Entre sus ventajas se cuenta una mayor productividad con un menor esfuerzo de programación. Esto es posible porque estas herramientas suelen contener bloques constructivos que engloban secuencias de instrucciones de uso muy habitual en diferentes aplicaciones (por ejemplo, el uso de objetos de interfaz gráfica de usuario). Estas herramientas además facilitan el despliegue, instalación y mantenimiento de la aplicación.

3.1.2 Según la forma de ejecución

Según como se realiza la ejecución de las instrucciones de un programa tenemos los siguientes tipos de lenguajes:

- ✓ Lenguajes compilados → Un programa traductor (denominado compilador) convierte el código fuente en código objeto y otro programa (denominado enlazador) unirá el código objeto del programa con el código objeto de las librerías necesarias para producir el programa ejecutable.
- ✓ Lenguajes interpretados → Ejecutan las instrucciones directamente, sin que se genere código objeto, para ello es necesario un programa intérprete en el sistema operativo o en la propia máquina donde cada instrucción es interpretada y ejecutada de manera independiente y secuencial. La principal diferencia con el anterior es que se

traducen a tiempo real solo las instrucciones que se utilicen en cada ejecución, en vez de interpretar todo el código, se vaya a utilizar o no.

- ✓ **Lenguajes virtuales** → Los lenguajes virtuales tienen un funcionamiento muy similar al de los lenguajes compilados, pero, a diferencia de éstos, no es código objeto lo que genera el compilador, sino un código intermedio o bytecode que puede ser interpretado por cualquier sistema que tenga la máquina virtual encargada de interpretar este bytecode generado para ejecutarlo; aunque de ejecución lenta (como los interpretados), tienen la ventaja de poder ser multiplataforma y así un mismo código bytecode sería válido para cualquier plataforma.

En un apartado posterior se verán conceptos asociados a la traducción de un programa, lo que redundará en un mejor entendimiento de estos tipos de lenguajes.

3.1.3 Según el paradigma de programación

Los paradigmas de programación son diferentes formas o estilos en los que un programa o lenguaje de programación pueden ser organizados. Cada paradigma consiste en ciertas estructuras y características sobre como los problemas de programación deberían ser abordados. Un paradigma debe resultar coherente y permitir responder a una visión particular de una forma de desarrollar.

Un paradigma es un modelo de programación y determina, por tanto, la formulación de algoritmos y, en consecuencia, la visión que tiene el desarrollador de la ejecución de su programa, así como la organización de su código fuente. Llevando este razonamiento hasta su extremo, podemos decir que la elección de paradigmas de un lenguaje de programación determina la forma de pensar, de analizar de un desarrollador y, en consecuencia, la forma de modelar los problemas a la hora de diseñar una solución.

Los paradigmas no son lenguajes o herramientas. Son más un conjunto de criterios y guías a seguir en el desarrollo de aplicaciones. Los lenguajes de programación no están atados a un paradigma concreto. Hay lenguajes contruidos bajo un cierto paradigma y tienen características que facilitan ese modelo de programación más que otros. Pero también hay lenguajes multi-paradigma lo que significa que el código se puede ajustar a un paradigma u a otro. Además, los paradigmas no son mutuamente exclusivos, en el sentido de que podemos usar prácticas de diferentes paradigmas a la vez en el desarrollo de una aplicación. Existen varios paradigmas de programación. Los siguientes son los más empleados.

- ✓ **Programación estructurada** → Propugna que la construcción de un programa solamente necesita sentencias de tipo secuencial, estructuras de control de flujo, bloques y subrutinas, para organizar mejor el código. Un programa escrito con solo tres tipos de sentencia (secuencial, selección y repetición) y dividido en subrutinas (procedimientos y funciones) resulta más fácil de leer, modificar y mantener.
- ✓ **Programación modular** → Consiste en dividir un programa en partes llamadas módulos con el fin de hacerlo más legible y manejable. Se presenta históricamente como una evolución de la programación estructurada para solucionar problemas de

programación más grandes y complejos de lo que esta puede resolver.

- ✓ Programación orientado a objetos → Los lenguajes orientados a objetos combinan en una estructura de programación, denominada objeto, tanto los datos como las funciones que operan sobre esos datos. Si se desea modificar los datos de un objeto, hay que realizarlo mediante las funciones miembro del objeto. Ninguna otra función puede acceder a los datos. Esto simplifica la escritura, depuración y mantenimiento del programa. En este paradigma, el programa se organiza como un conjunto de objetos que contienen datos y operaciones que procesas esos datos. Cuando se trata de resolver un problema, este se descompone en objetos. El pensar en términos de objetos tiene una gran ventaja: se asocian los objetos del problema a los objetos del mundo real.

3.2 Traducción de código fuente

Para crear una aplicación, lo más habitual es utilizar un lenguaje de 3ª generación o superior. En este caso el programa creado por el programador se tiene que traducir a lenguaje máquina para que el ordenador pueda ejecutarlo. Se deduce por tanto que cuando se desarrolla un programa o aplicación vamos a tener dos versiones del mismo. Uno está escrito en el lenguaje de programación en cuestión y es el que maneja el programador. El otro está expresado en lenguaje máquina y ejecutable por el ordenador.

Código fuente son todas las sentencias de una aplicación escritas en un determinado lenguaje de programación.

Código máquina o ejecutable son las instrucciones de la misma aplicación escritas en lenguaje máquina o binario que el ordenador puede ejecutar y son específicas de una plataforma.

Gráficamente sería así:



Figura 5.- Proceso de traducción

Cuando se traduce código fuente en código máquina las instrucciones generadas en lenguaje máquina son específicas de un tipo concreto de procesador. Por tanto el resultado solamente puede ejecutarse en una plataforma específica.

Plataforma es un sistema, definido por una arquitectura hardware y un sistema operativo, que sirve como base para hacer funcionar un determinado software con el que es compatible.

Por ejemplo, cuando hablamos de una plataforma IBM-PC podemos referirnos a un ordenador con un procesador x86 o AMD64. Una plataforma Windows incluiría un sistema anterior donde se ejecuta un sistema operativo Windows.

Generalmente, la traducción de código fuente a código máquina es realizada por una aplicación, la cual puede adoptar las siguientes formas:

3.2.1 Intérprete

Es una aplicación que traduce un programa en código fuente sentencia a sentencia. Cada vez que traduce una sentencia, la ejecuta a continuación. Así sucesivamente hasta que termina la ejecución del programa. La desventaja es la lentitud, porque para ejecutar un programa hay que traducir cada sentencia antes de ejecutarla. Además, el programa en código fuente es imprescindible para su ejecución y cada plataforma dispone de su propio intérprete para poder ejecutar la aplicación.

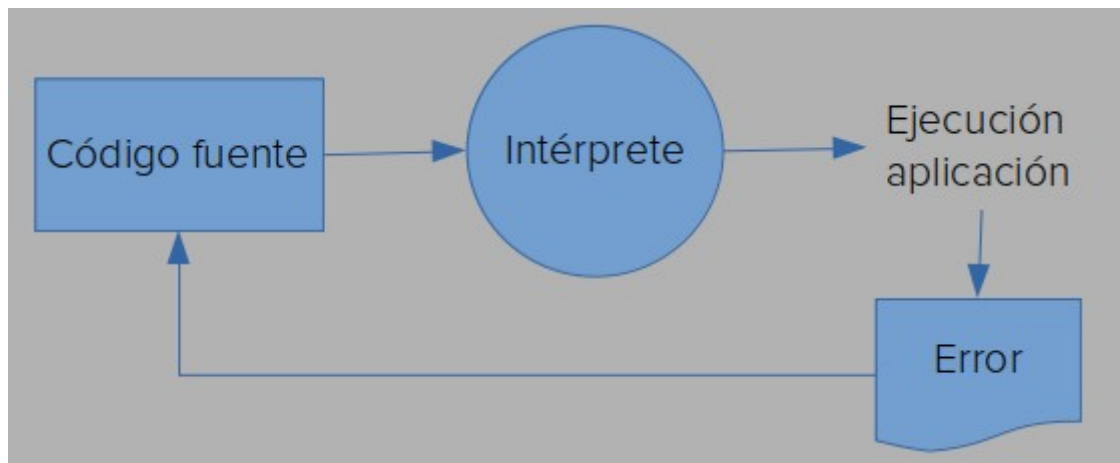


Figura 6: Esquema de interpretación

3.2.2 Compilador

Es una aplicación que traduce un programa escrito en código fuente completo a código máquina. El resultado es un programa en código máquina directamente ejecutable en el ordenador. La ventaja respecto al anterior es que primero se realiza la traducción completa y luego se ejecuta, con lo que la ejecución es más rápida. La desventaja es que el programa ejecutable generado es para una plataforma en particular, no pudiéndose ejecutar en un ordenador con otra diferente.

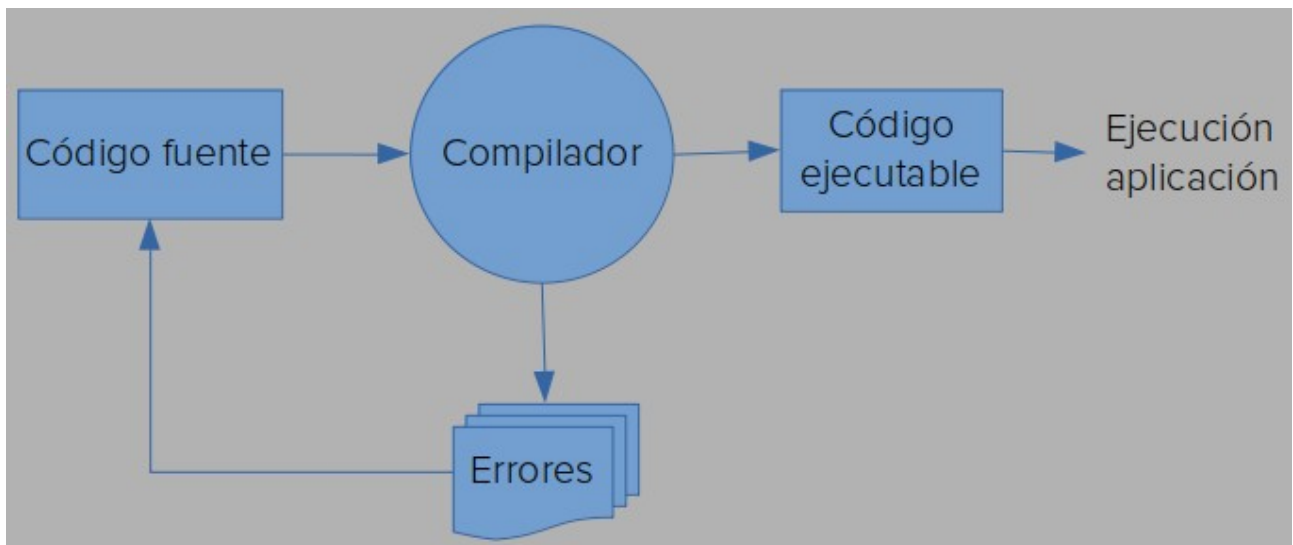


Figura 7: Esquema de compilación

El código compilado es directamente ejecutable por la máquina, sin embargo, se necesita un proceso más para que tengamos un programa completamente funcional.

3.2.3 Código intermedio y máquina virtual

Consiste en una combinación de los dos anteriores. El programa escrito en código fuente se compila dando como resultado un programa escrito en código intermedio denominado bytecode. Posteriormente, un intérprete ejecuta el código intermedio generado. Uno de los lenguajes más populares que existen, Java, utiliza este método de traducción. La ventaja radica en que un mismo programa compilado a bytecode puede ejecutarse en diferentes plataformas mediante el uso de una máquina virtual específica en cada plataforma que incluye la función de intérprete.

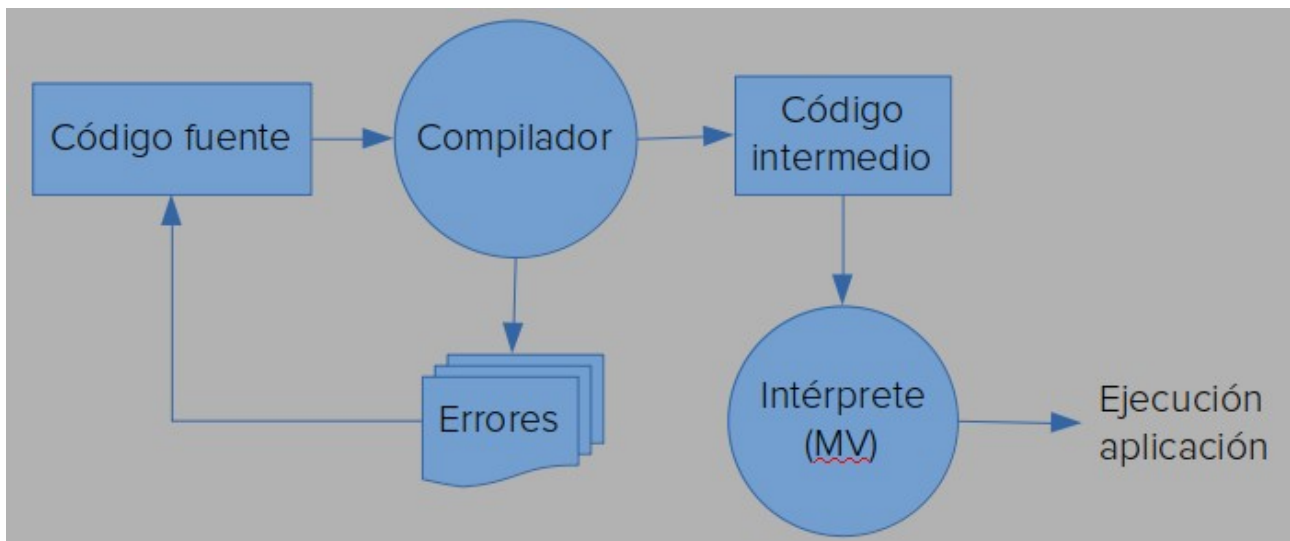


Figura 8: Esquema de código intermedio

En este último caso, la ejecución del software desarrollado no depende de un hardware y sistema operativo específico, sino del intérprete que genera el código máquina desde el código intermedio para su ejecución. Por tanto, la plataforma vendría dada por la máquina virtual específica del lenguaje.

Por ejemplo, cuando hablamos de que una aplicación se ejecuta en una plataforma Java estamos diciendo que la aplicación está escrita en lenguaje de programación Java y se ha traducido a código intermedio para ser ejecutado por la máquina virtual de Java, la cual deberá estar instalada en el ordenador donde queremos ejecutar la aplicación.

Dado que el código máquina está ligado al hardware de la máquina que lo ejecuta, ese código no es portable (trasladable) entre ordenadores de diferente tipo. Los programas escritos en lenguajes de alto nivel pueden ser trasladados entre distintas máquinas usando un intérprete diferente en cada una de ellas, o recompilando el código para crear una versión diferente del código máquina del programa para cada uno de los tipos de ordenador.

3.2.4 Biblioteca

Una biblioteca o librería (*library*) es un conjunto de funciones codificadas en un lenguaje de programación que ofrecen una funcionalidad concreta y de uso común en múltiples aplicaciones. La biblioteca se puede incorporar al desarrollo de una aplicación para ahorrar tiempo y costes.

Las bibliotecas facilitan el desarrollo de aplicaciones al programador ya que al utilizarlas en sus proyectos en desarrollo se ahorran la escritura de código para implementar las funciones que incluye la biblioteca. Las ventajas de emplear bibliotecas son:

- ✓ Las funciones de la biblioteca se escriben una vez y se emplean muchas veces, lo que permite reutilizar código ya escrito, probado y depurado.

- ✓ El código de una biblioteca puede haberse escrito por un programador y utilizarse por múltiples programadores lo que promueve el desarrollo colaborativo.
- ✓ Cada función de la biblioteca tiene una interfaz claramente definida y documentada para que el programador pueda utilizarla.

A diferencia de una aplicación, una biblioteca por si sola no es ejecutable, sino que su fin es ser utilizada por otras aplicaciones, independientes y de forma simultánea. Unas bibliotecas pueden requerir de otras para funcionar, e incluso puede haber bibliotecas que refinan o alteran el comportamiento de una biblioteca original. Por tanto, las bibliotecas pueden vincularse a una aplicación o a otra biblioteca en distintos puntos del desarrollo o la ejecución de la aplicación.

Los programadores no son los únicos que pueden desarrollar bibliotecas para sus proyectos o para terceros. La mayoría de los sistemas operativos modernos también proporcionan bibliotecas que implementan los servicios del sistema. Además, los lenguajes de programación suelen incluir funciones integradas, las cuales se encuentran en lo que se conoce como bibliotecas estándar del lenguaje. Para diferenciarlas de las que crean los programadores y ponen a disposición de otros programadores, estas se conocen como bibliotecas de terceros.

Una biblioteca puede contener código fuente, código máquina o bytecode. En cada caso podría utilizarse en programas escritos con el mismo lenguaje utilizado al desarrollar la biblioteca. Sin embargo, hay casos de programas escritos en un lenguaje y que utilizan bibliotecas desarrolladas en otro lenguaje.

Hay dos tipos de bibliotecas:

- ✓ Estáticas → Es un fichero contenedor con varios archivos de código objeto empaquetados, que en el proceso de enlazado durante la compilación serán incorporados en el fichero ejecutable final, junto con el resto de ficheros de código objeto de la aplicación.

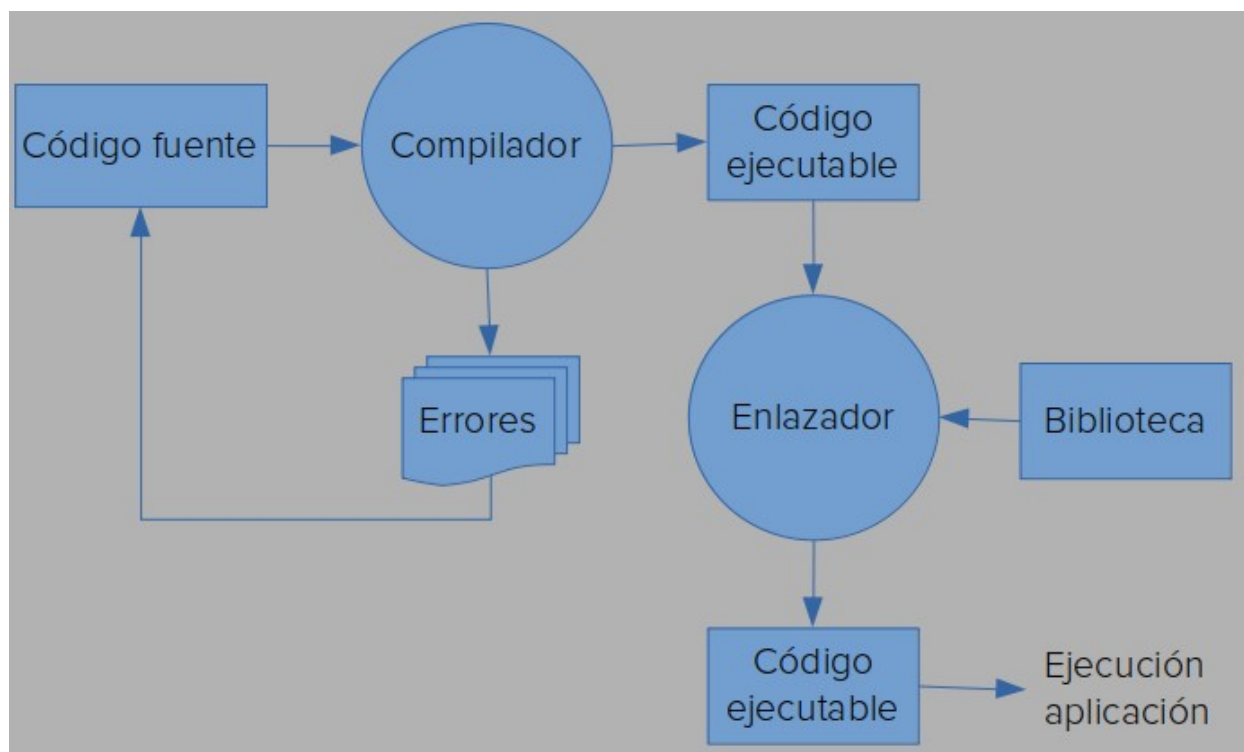


Figura 9: Enlace estático de bibliotecas

- ✓ **Dinámicas** → Son ficheros que contienen código objeto construido de forma independiente de tal modo que están preparadas para poder ser requeridas y cargadas en memoria durante la ejecución de cualquier programa, en lugar de tener que ser enlazadas, previamente, en tiempo de compilación. Por tanto, han de estar disponibles como ficheros independientes al programa ejecutable (generalmente en directorios del sistema). En el proceso de enlazado (en tiempo de compilación) se genera un fichero ejecutable con anotaciones de qué bibliotecas dinámicas requiere e información añadida para el cargador dinámico, el cual se encargará de cargar en memoria la función de la biblioteca cuando se invoque desde la aplicación una de las funciones de la biblioteca.

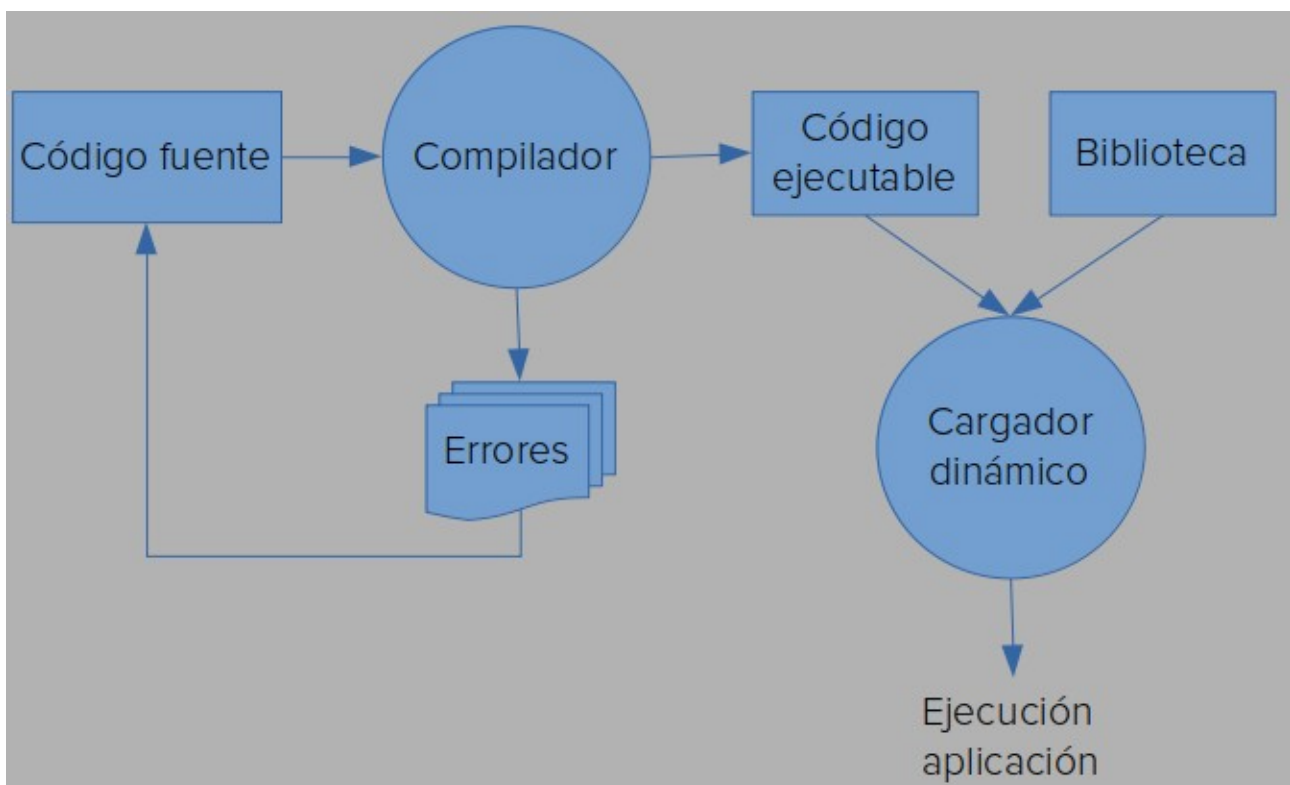


Figura 10: Enlace dinámico de biblioteca

3.3 Código fuente, objeto y ejecutable

El código de un programa es manejado con programas externos comúnmente asociados al lenguaje de programación en el que está escrito nuestro programa, y a la arquitectura en donde queremos ejecutar dicho programa. Del apartado anterior podemos deducir que un programa está formado por código que pasa por diferentes estados antes de poder ser ejecutado. Estos son:

- ✓ **Código fuente** → El código fuente de un programa informático es un conjunto de sentencias escritas en un lenguaje de programación determinado. Es decir, es el código en el que los programadores escriben los programas.
- ✓ **Código objeto** → El código objeto es el código resultante de compilar el código fuente. Si se trata de un lenguaje de programación compilado, el código objeto será código máquina, mientras que si se trata de un lenguaje de programación virtual, será código bytecode.
- ✓ **Código ejecutable** → El código ejecutable es el resultado obtenido de enlazar nuestro código objeto con las librerías. Este código ya es nuestro programa ejecutable, programa que se ejecutará directamente en nuestro sistema o sobre una máquina virtual en el caso de los lenguajes de programación virtuales.

Cabe destacar que, si nos encontrásemos programando en un lenguaje de programación interpretado, nuestro programa no pasaría por el compilador y el enlazador, sino que solo tendríamos un código fuente que pasaría por un intérprete que realizaría la

traducción y ejecución línea a línea en tiempo real. En este caso, las bibliotecas que pudiera utilizar el programa, también estará en código fuente.

4 Desarrollo de aplicaciones

Crear un software no es solo escribir código. El desarrollo de una aplicación consta de una serie de tareas relacionadas entre si cuyo objetivo es, partiendo de la necesidad de solucionar un problema de gestión de información, implementar una solución a ese problema mediante una aplicación software.

Un proceso de desarrollo de software es la descripción de una secuencia de actividades que deben ser seguida por un equipo de personas para generar un conjunto coherente de productos, uno de los cuales es una aplicación software.

En aplicaciones de tamaño medio y grande, el proceso de desarrollo es llevado a cabo por un equipo de profesionales, que pueden estar agrupados a la hora de realizar una determinada tarea, trabajando de forma coordinada y en un ambiente colaborativo.

El objetivo de un proceso de desarrollo software es la formalización de las actividades relacionadas con el desarrollo de aplicaciones de un sistema informático para estimar su coste, plazo y calidad del software.

4.1 Ciclo de vida del software

La mayoría de los proyectos software no cumplen plazos y exceden el presupuesto asignado. Debido a ello, todo desarrollo software necesita de una planificación previa en la que se definen las tareas a realizar, el equipo necesario y el plazo para obtener el producto final. Una definición concisa del ciclo de vida es:

El ciclo de vida de una aplicación software es el periodo de tiempo que transcurre desde que surge la necesidad de dicha aplicación hasta que deja de utilizarse.

No hay una única forma de desarrollar software. Existen varios modelos de desarrollo que especifican como se lleva a cabo el proceso de crear un nuevo software, cada uno con ventajas e inconvenientes, escogiendo aquél que se ajuste mejor al tipo de software que se desarrolla. En todos ellos, el ciclo de vida de una aplicación se divide en varias fases que pueden ejecutarse en serie o en paralelo, dependerá de la metodología utilizada. Generalmente estas fases son:

4.1.1 Planificación

En esta fase se trazan objetivos y se valoran las necesidades del proyecto en cuestión. Las tareas iniciales consisten en realizar un estudio de viabilidad, analizar los riesgos que puede comportar el proyecto y planificar todas las fases posteriores.

- ✓ El estudio de viabilidad nos permite evaluar los aspectos técnicos, financieros y legales, para poder determinar si el proyecto podrá ser completado con los recursos

disponibles.

- ✓ Analizar los riesgos nos puede ayudar a estar preparados para cualquier contratiempo.
- ✓ Planificar el proyecto fase por fase nos permite llevar a cabo su desarrollo con éxito.

A continuación, se documentan en detalle las funcionalidades, características y objetivos que debe cumplir el ciclo de vida del proyecto.

4.1.2 Análisis

La fase de análisis define los requisitos del software que hay que desarrollar. Inicialmente, esta etapa comienza con una entrevista al cliente, que establecerá lo que quiere o lo que cree que necesita, lo cual nos dará una buena idea global de lo que necesita, pero no muy precisa y detallada. Aunque el cliente crea que sabe lo que el software tiene que hacer, es necesaria una buena habilidad y experiencia para reconocer requisitos incompletos, ambiguos, contradictorios o incluso innecesarios. Es importante que en esta etapa del proceso de desarrollo se mantenga una comunicación bilateral, aunque es frecuente encontrarse con que el cliente pretenda que dicha comunicación sea unilateral, es necesario un contraste y un consenso por ambas partes para llegar a definir los requisitos verdaderos del software.

El resultado final será un documento formal denominado *Especificación de Requisitos del Software* el cual establece claramente las necesidades que debe satisfacer el software que se desarrolla teniendo en cuenta los requisitos del usuario final.

4.1.3 Diseño

Consiste en construir una especificación de un software tomando como punto de partida los requisitos que tiene que satisfacer. En esta etapa se pretende determinar el funcionamiento de una forma global y general, sin entrar en detalles. Uno de los objetivos principales es establecer las consideraciones de los recursos del sistema, tanto físicos como lógicos.

Esta etapa está dividida en varias partes las cuales abordan el diseño de la aplicación en varios aspectos de la aplicación:

- ✓ Diseño de datos → Se diseña la base de datos que almacenará la información de la aplicación.
- ✓ Diseño Funcional → Se crearán los diagramas de casos de uso y de secuencia para definir la funcionalidad del sistema.
- ✓ Interfaz de usuario → Diseñar un entorno intuitivo y fácil de usar, es otro elemento clave para un proyecto de desarrollo de software. Que el usuario sea capaz de acceder a los datos y gestionar la aplicación de manera ágil, es un requisito indispensable.

4.1.4 Codificación

En esta fase se codifica el código de la aplicación con el lenguaje de programación elegido. En esta fase es cuando se realiza la programación de la aplicación.

Gracias a las etapas anteriores, el programador contará con un análisis completo del sistema que hay que codificar y con una especificación de la estructura básica que se necesitará, por lo que en un principio solo habría que traducir el resultado del diseño en una solución implementada en el lenguaje de programación deseado. Sin embargo, las dificultades son recurrentes mientras se codifica. Por supuesto que cuanto más exhaustivo haya sido el análisis y el diseño, la tarea será más sencilla, pero nunca está exento de necesitar una redefinición del análisis o del diseño al encontrar un problema durante la codificación.

Es fundamental escoger el lenguaje de programación adecuado a nuestro proyecto y disponer de un equipo de programadores expertos. Seguir el código de buenas prácticas permitirá que nuestro proyecto sea fácilmente escalable.

4.1.5 Pruebas

Una vez terminado el proceso de codificación empieza la fase de pruebas de la aplicación. En esta etapa el software desarrollado se somete a pruebas para verificar su correcto funcionamiento y que se ajusta a los requisitos establecidos en la fase de análisis. Es una fase de corrección, eliminación y perfeccionamiento de posibles fallos, no previsto en los pasos previos.

4.1.6 Despliegue o instalación

El despliegue de software es el proceso de entregar aplicaciones de software desde entornos de desarrollo a producción, poniéndolas a disposición de los usuarios finales de forma segura, confiable y eficiente. Debe garantizar que el software se configure, pruebe, lance e instale de forma coherente y controlada.

El despliegue de software resulta esencial para garantizar que las aplicaciones se entregan e instalan de forma correcta y eficaz. Esta práctica ayuda al equipo de desarrollo a proporcionar una aplicación de alta calidad que satisfaga las necesidades de los usuarios finales.

Para garantizar el éxito resulta esencial contar con un proceso bien definido y documentado que incluya todos los pasos requeridos y las mejores prácticas.

4.1.7 Documentación

Por norma general, la documentación que se realiza de un software tiene dos caras: la documentación disponible para el usuario final del software y la documentación destinada al propio equipo de desarrollo.

La documentación para el usuario debe mostrar una información completa y de calidad que ilustre mediante los recursos más adecuados cómo manejar la aplicación. Una buena documentación debería permitir a un usuario cualquiera comprender el propósito y el modo

de uso de la aplicación sin información previa o adicional.

Por otro lado, tenemos la documentación técnica, destinada a equipos de desarrollo, que explica el funcionamiento interno del programa, haciendo especial hincapié en explicar la codificación del programa. Se pretende con ello permitir a un equipo de desarrollo cualquiera poder entender el programa y modificarlo si fuera necesario. En casos donde el software realizado sea un servicio que pueda interoperar con otras aplicaciones, la documentación técnica hace posible que los equipos de desarrollo puedan realizar correctamente el software que trabajará con nuestra aplicación.

4.1.8 Mantenimiento

Generalmente, todo el software necesita un mantenimiento continuo. En esta fase del desarrollo de un software se arreglan los fallos o errores que suceden cuando el programa ya ha sido implementado en un sistema. Además, se realizan las ampliaciones necesarias o requeridas.

Una vez el software está en explotación se adapta a nuevas necesidades, se amplía su funcionalidad y se corrigen errores no detectados durante la fase de pruebas o introducidos nuevos.

En este periodo el software ya está en funcionamiento. Con el tiempo alguna función puede quedar obsoleta, pueden detectarse algunas limitaciones o que aparezcan propuestas que mejoren la estabilidad del proyecto.

Cuando el mantenimiento que hay que realizar consiste en una ampliación, el ciclo de vida suele volverse cíclico, por lo que, dependiendo de la naturaleza de la ampliación, puede que sea necesario analizar los requisitos, diseñar la ampliación, codificar la ampliación, probarla, documentarla, implementarla y, por supuesto, dar soporte de mantenimiento sobre la misma, por lo que al final este modelo es recursivo y cíclico para cada aplicación y no es un camino rígido de principio a fin.

4.2 Herramientas de desarrollo

Para desarrollar aplicaciones, los programadores cuentan con un conjunto de herramientas las cuales son aplicaciones que facilitan, y en algunos casos automatizan, las fases del desarrollo de una aplicación, principalmente la codificación (escritura de código), traducción y despliegue.

- ✓ Editor de código → Un editor de código fuente es un editor de texto diseñado específicamente para escribir el código fuente de programas informáticos. Puede ser una aplicación individual o estar incluido en un entorno de desarrollo integrado. Los editores de código fuente tienen características diseñadas exclusivamente para simplificar y acelerar la escritura de código fuente, como resaltado de sintaxis, autocompletado y emparejamiento de llaves, corchetes y paréntesis. Estos editores también proveen un modo de ejecutar un compilador, un intérprete, un depurador, o cualquier otro programa que sea relevante en el proceso de desarrollo de software.
- ✓ Compilador o intérprete → Consiste en el traductor de código fuente a código

máquina o intermedio, en función del tipo de lenguaje elegido. En el caso de los intérpretes también ejecutan la instrucción ejecutada.

- ✓ Depurador → Un depurador (*debugger*) es un programa utilizado para probar y depurar (eliminar) los errores de otros programas. Algunos errores en el software pueden ser descubiertos durante su ejecución normal, pero hay otros que son más difíciles de ver y encontrar con una ejecución del programa. En estos casos se hace imprescindible el uso de un depurador. Entre sus funciones está el poder ejecutar un programa paso a paso para inspeccionar su funcionamiento o definir puntos de ruptura a lo largo del código a examinar para que se ejecución se detenga en cada uno de ellos y, así, el programador puede inspeccionar el estado de la ejecución del software en un punto concreto. También, puede ejecutar el software y detenerse cuando se encuentra una condición específica.
- ✓ Pruebas → Consiste en ejecutar el programa para detectar errores o defectos en el software. Cuanto más se pruebe un software menor será la probabilidad de que contenga errores. Por ello es necesario realizar ejecuciones con diferentes datos de entrada y que recorran todos los posibles flujos de ejecución de instrucciones, comprobando que los datos de salida son los realmente esperados. Debido a que la ejecución de múltiples pruebas puede ser un proceso lento y tedioso existen herramientas que permiten automatizarlas.
- ✓ Generador de documentación → Es una herramienta que genera documentación automáticamente destinada a los programadores (en el caso de la documentación de la API) o a usuarios finales (en el caso del manual de usuario), o a ambos, a partir de un código fuente al que se ha añadido comentarios especiales con el propósito de generar automáticamente su documentación.
- ✓ Sistema de control de versiones → Se llama control de versiones a la gestión de los diversos cambios que se realizan sobre los elementos de algún software o una configuración del mismo. Una versión, revisión o edición de un software, es el estado en el que se encuentra el mismo en un momento dado de su desarrollo o modificación. Aunque un sistema de control de versiones puede realizarse de forma manual, es muy aconsejable disponer de herramientas que faciliten esta gestión dando lugar a los llamados sistemas de control de versiones. Estos sistemas facilitan la administración de las distintas versiones de cada producto software desarrollado, así como las posibles especializaciones realizadas para una implementación específica.

5 Bibliografía

OLIARTE, L. (2018, 23, 4) Lenguaje de Programación. *Conogasi*:
<https://conogasi.org/articulos/lenguaje-de-programacion/>

RAMOS, M. (s.f) Sintaxis, Semántica y Gramática del Lenguaje de Programación. *Web Creator*:
<http://manuelaliramos.com/lenguajes-de-programacion/sintaxis-semantica-gramatica-del-lenguaje-programacion/>

Evotic. (s.f.). Ciclo de vida del software. Etapas y modelos del Ciclo de Vida de un Software.
<https://evotic.es/software-a-medida/ciclo-de-vida-del-software/>